

Summer Report

Overview and Purpose

The lateral line is a bio-inspired project based on the biology of fish. Fish can immediately sense the flow around them using a lateral line, a distributed array of sensory organs running along the body. It supports schooling, obstacle avoidance, and predation in water, all without relying on sight, which can help in dark or murky conditions where vision fails. The project hopes to imitate the sensory organs called canal neuromasts, using differential pressure sensors that each measure across two ports.

Water flow moving relative to a body creates a pressure distribution on the surface, so an array of pressure measurements should be able to measure a flow field. The lateral-line system aims to record the flow of water and disturbances in a predictable manner. Steady flow of water over a body produces a smooth and largely predictable pressure pattern, so disturbances would be thought to indicate something about the surrounding conditions. Unsteady flow can produce different pressure patterns. When water passes a blunt object, it separates and sheds into a vortex pattern downstream. The difference in vortex is assumed to change based on how large the object is and how fast the water is moving.

One of the applications the lateral line can contribute to is wake detection: identifying the presence and approximate size of an upstream figure from a vortex pattern that sheds. Even further, closed loop vehicle control can benefit from the system, recording hydrodynamic force in real time through a sensing array.

This report documents the lateral line board as it stands August 2026, covering the hardware, firmware, and processing for the lateral line.

Summer Report

Lateral Line Sensory System

Schematic and Prototype

The pre-existing PCBs could not distribute signals effectively, so a new design was necessary. The new approach includes an ESP32 that takes in signals from eight MPXV7002 Differential Sensors. Due to the output voltage from the sensors exceeding 3.3 V, a voltage divider was needed to scale down the sensor output voltage.

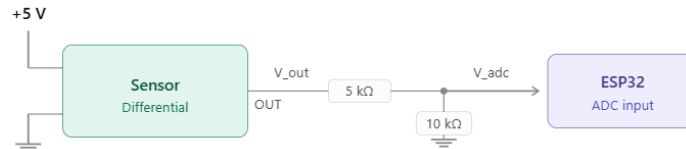


Figure 1: Singular Sensor Layout

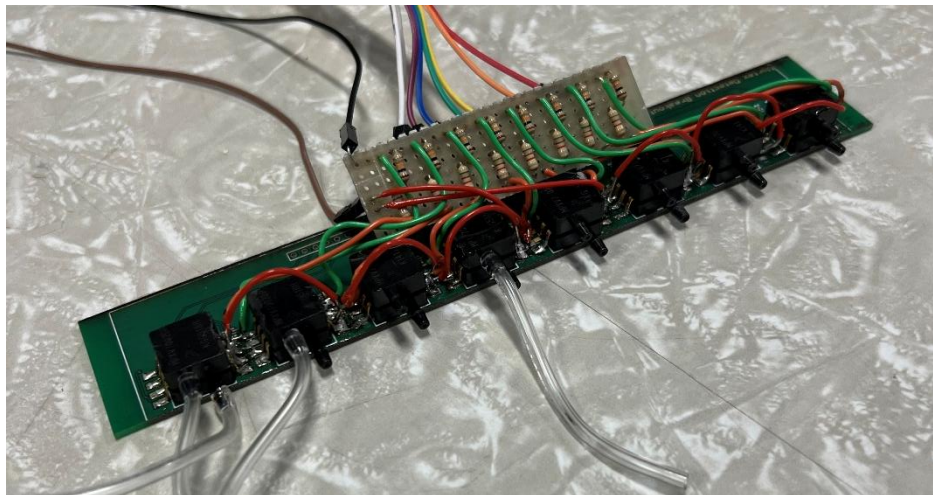


Figure 2: Prototype Board

Firmware

The firmware on the ESP32 utilizes GPIO pins that are connected to ADC channels to receive the output signal from the differential sensors. The firmware is a simple program indicated with Differential-Sensor/src/main.cpp that primarily serves to take in the voltage signal from the sensors and then output the pressure values of all eight sensors.



Figure 3: Flow of Data

Summer Report

The data is sent towards the computer in a format that includes the timestamp and then all of the pressure sensor values are separated by a comma.

time, pressure[0], pressure[1], pressure[2] ...

As a result, the python script must process the data in this precise format to ensure that all the data is being collected properly. To program the ESP32, it is necessary to press the boot button on the ESP32. This is not possible without having access to the board while covered in silicone. There was also calibration done on each of the sensors by manually zeroing the pressure based on a known pressure value. This is altered through a struct in the main.cpp file.

Python Script

The ESP32 streams one line of text per sample over the serial connection, giving a timestamp in milliseconds followed by a pressure reading in kilopascals for each of the eight channels. A Python script on the host computer reads this stream, and it can be run in one of two modes.

In live mode, the script opens a window that updates as data arrives and plots all eight channels at once, scrolling to keep a fixed window of recent time on screen. This mode is meant for checking that the sensors are working and for watching a signal as it happens, and it runs until the window is closed.

In export mode, the script records for a set duration, then saves the run as two files: a CSV containing every sample and its timestamp, and a PNG showing all eight channels plotted together. Each run is saved with the date and time in its filename, so repeated runs do not overwrite one another.

The mode, filtering, serial port, and recording length are all set at the top of the script, so switching between live viewing and recording a run does not require touching the rest of the code.

The filter works by moving the signal from the time domain into the frequency domain, removing noise, and moving it back. `np.fft.rfft` takes the recorded pressure trace and decomposes it into the sine waves that sum to produce it, each with its own frequency and amplitude. `np.fft.rfftfreq` labels which frequency each of those components corresponds to, based on the sample spacing. Every component above `FILTER_CUTOFF_HZ` gets its amplitude set to zero, and `np.fft.irfft` reconstructs a time-domain signal from what's left. The result is a version of the same trace with high-frequency content removed acting as a ideal low-pass filter. This signal processing is very beginner just to smooth out signals but further work and investigation into techniques to achieve the desired signaling will need to be done.

```
# Configuration
# =====

MODE = "export"           # "live" or "export"

FILTER_MODE = False       # low-pass the pressure traces
FILTER_CUTOFF_HZ = 5.0    # discard content above this frequency

RECORDING_SECONDS = 5     # export mode only
DATA_DIRECTORY = "Data"  # export mode only

PORT = "COM4"
BAUD = 115200
CHANNELS = 8

# =====
```

Figure 4: Configuration File of Python Script

Summer Report

Pitot Tube

A pitot tube measures fluid velocity by comparing the pressure at its facing tip, where flow is brought to rest, against a reference pressure. It is used here to try and create a independently trusted method to identify the velocity of the water moving in the tank before the sensors are fully used in the lateral line.

The pitot tube only utilizes one differential sensor and uses similar firmware and software to collect data. The main difference is that the firmware includes the formula to calculate velocity from pressure.

Cross-section profiling.

The tank cross section was mapped at 25 points on a 5×5 grid (5 heights × 5 distances from the inner wall) at 300 RPM and 8.5 in water depth, to find the velocity profile across the channel. Still water alone read a nonzero, non-repeatable velocity around 0.1- 0.2 m/s, and velocity continued to decrease gradually even with the motor running and the tube held in a fixed position, eventually settling toward the still-water noise floor. Resolving that drift required taking repeated 30-second windows at the same point until the dropoff rate stabilized. The resulting profile isn't what a developed channel flow should look like: it doesn't peak near the center and taper toward the walls, but instead varied with height, not with the barriers of the tank.

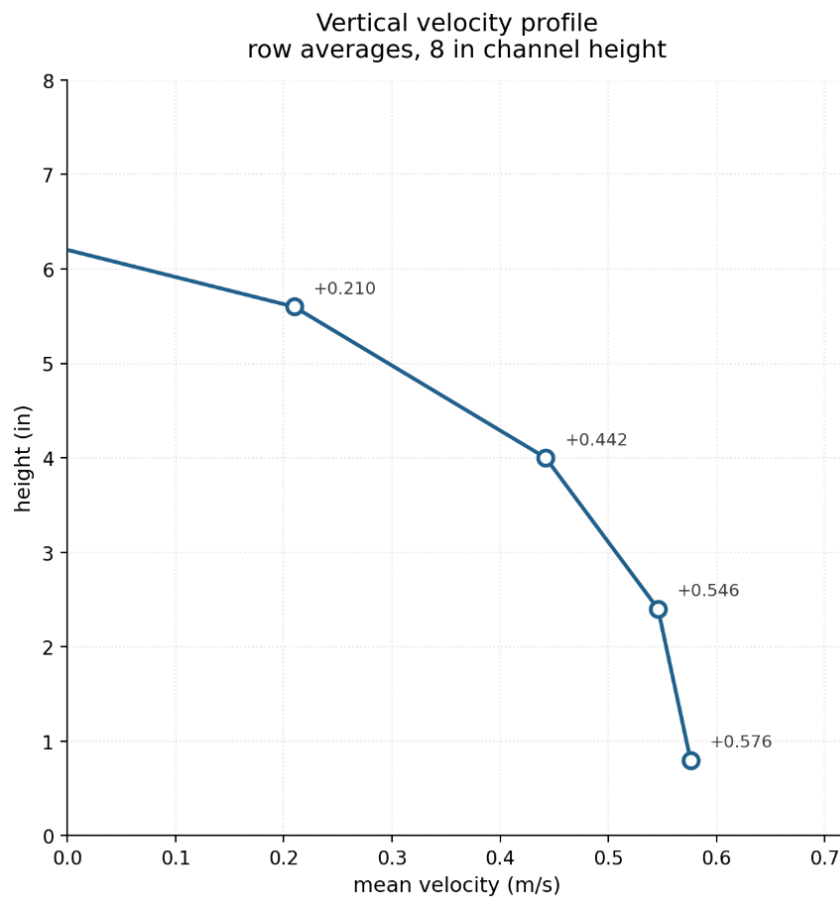


Figure 5: Vertical Velocity Profile

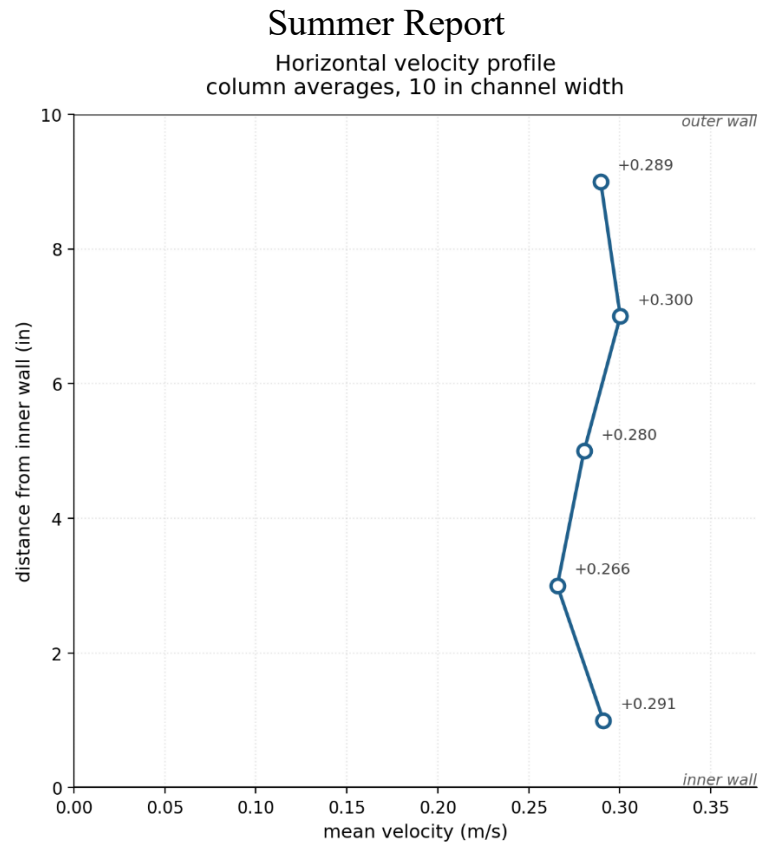


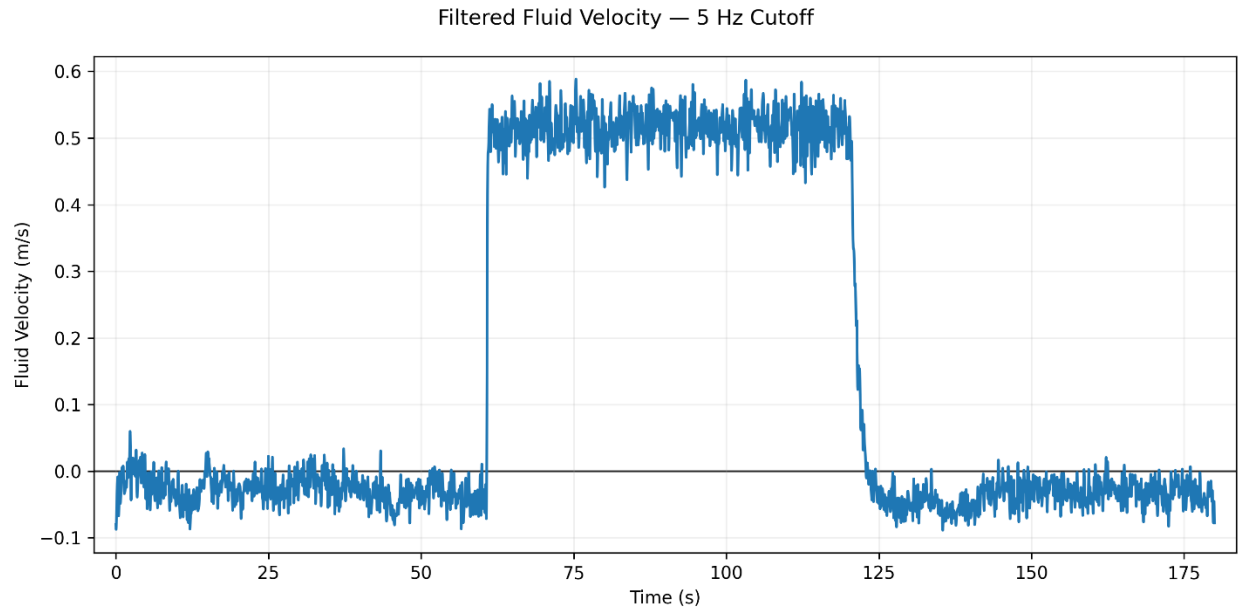
Figure 6: Horizontal Velocity Profile

The results of these tests proved that the experimental setup needed to be fixed for the pitot tube so that the data could be validated. At rest or constant velocity, the measurements were significantly off. As a result, the experimental process needed to be fixed variable by variable to produce repeatable results.

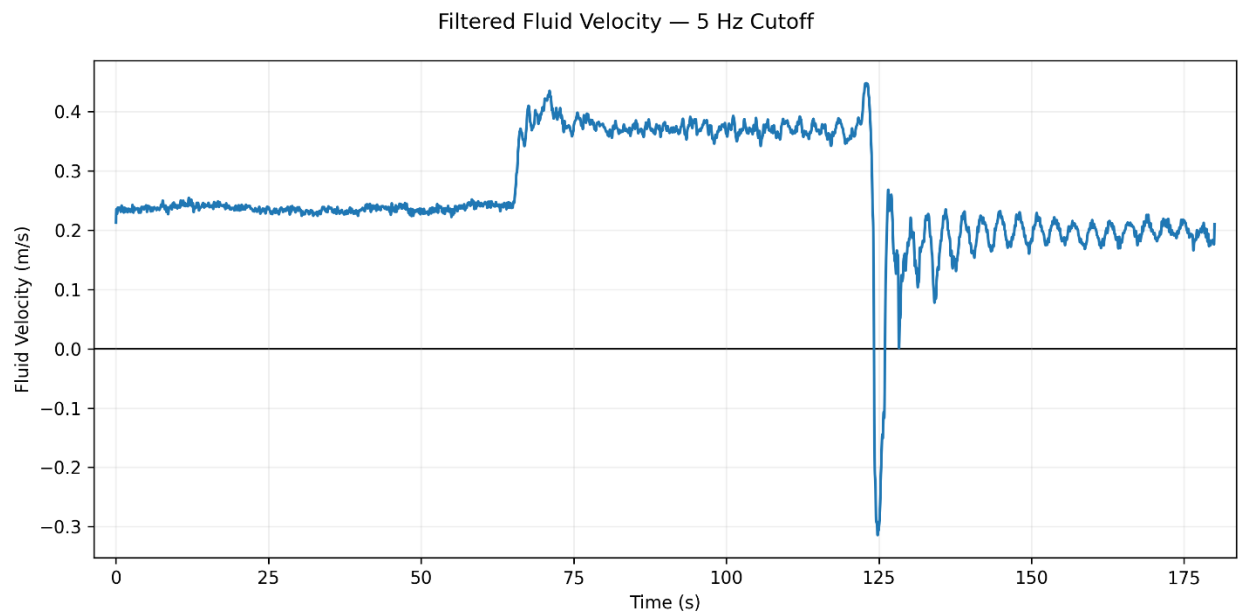
Summer Report

Isolate the sensor from the tank.

To separate sensor behavior from tank behavior, the same on/off test was run first in air using a vacuum, then in water, cycling the tube through rest, flow, and then rest. In air, the sensor gave a clean step up and back down. In water, securing the tubing (which had been letting air in during the air-to-water transition) improved the signal drastically which shows a large step towards accurate signaling.



The sensor predictably increased when the motor was on, but then decreased when the motor was off. What these tests tell us is that the sensor and setup has some reliability to detect changes in velocity, but there is something different when using the water. We then replicated the test but in water to try and produce a clean signal like what was done in air.



Darren Leong
August 18, 2026

Summer Report

Previous tests we saw the gradual decrease in the velocity even when the motor was running. We believe securing the tubing did help mitigate this effect. The sensor setup is slightly improved, but we still **see the wave formation with the membrane at the tank**. These tests show that we need a specific method for consistency with the sensor since many factors can change the data as the testing on 8-4 showed.

Future Work

Complete the velocity profile and research the water tank behavior to get accurate velocity measurements

Start integrating the lateral line